NExSys: Naturally Expressive Systems Engineering through Contrastive Learning for SysML2-Language Joint Embeddings

Anonymous Authors[1]

Abstract

Systems Modeling Language v2 (SysML2) represents a critical advancement in model-based systems engineering, providing a rigorous framework for specifying complex cyber-physical systems. However, the semantic gap between formal SysML2 specifications and natural language descriptions poses significant challenges for accessibility, automation, and cross-domain collaboration. We present **NExSys** (**N**aturally **Ex**pressive **Sys**tems Engineering), a novel framework for learning joint embeddings between natural language and SysML2 through contrastive learning. Leveraging the Qwen family of encoder-only and encoder-decoder models, NExSys learns a shared embedding space that captures both the structural semantics of SysML2 and the distributional semantics of natural language. We curate a comprehensive dataset of thousands of aligned SysML2-natural language pairs spanning diverse domains including aerospace, automotive, robotics, and enterprise systems. NExSys enables bidirectional translation, semantic search, and requirements traceability through its hierarchical contrastive learning objectives that operate at element, component, and system levels. Extensive experiments demonstrate that NExSys achieves state-of-the-art performance on SysML2-to-text generation (BLEU: XX.X), text-to-SysML2 synthesis (exact match: XX%), and cross-modal retrieval tasks (Recall@10: XX.X%). This work opens new avenues for AI-assisted systems engineering, automated documentation generation, and natural language interfaces for formal modeling tools.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Model-based systems engineering (MBSE) has emerged as the cornerstone methodology for developing complex cyber-physical systems, from autonomous vehicles to space exploration missions (**??**). At the heart of MBSE lies the Systems Modeling Language (SysML), recently evolved into its second version (SysML2) (**?**), which provides unprecedented expressiveness for capturing system requirements, behavior, structure, and parametric constraints. Despite these advances, a fundamental challenge persists: the semantic chasm between formal modeling languages and natural language communication among stakeholders.

This gap manifests across multiple dimensions of systems engineering practice. Requirements engineers struggle to translate stakeholder needs expressed in natural language into precise SysML2 specifications. System architects face difficulties in generating human-readable documentation from formal models. Project managers require accessible summaries of complex system behaviors without navigating intricate model hierarchies. These challenges are amplified in modern systems engineering contexts where interdisciplinary teams span traditional engineering boundaries, incorporating software developers, machine learning practitioners, domain experts, and business stakeholders who may lack formal modeling expertise.

Recent advances in foundation models for natural language processing have demonstrated remarkable capabilities in understanding and generating human language across diverse domains (**???**). Simultaneously, progress in multimodal learning has shown that shared embedding spaces can bridge disparate modalities—vision and language (**?**), code and natural language (**??**), and structured data representations (**?**). However, domain-specific formal languages like SysML2, which combine textual syntax, graphical semantics, and mathematical constraints, present unique challenges not addressed by existing multimodal frameworks.

We introduce **NExSys** (**N**aturally **Ex**pressive **Sys**tems Engineering), a novel contrastive learning framework for learning joint embeddings between natural language and

SysML2. NExSys leverages the Qwen family of models (**?**)—specifically encoder-only variants for efficient embedding extraction and encoder-decoder architectures for generative tasks. The NExSys framework is grounded in three key insights:

First, SysML2's textual representation provides a natural interface for sequence-to-sequence learning, while its formal semantics enable rigorous evaluation of structural correctness. Unlike purely graphical modeling languages, SysML2's textual syntax allows direct application of transformer-based architectures while maintaining the precision of formal specification languages.

Second, contrastive learning objectives naturally align with the requirements traceability problem in systems engineering, where the fundamental task is to identify semantic correspondence between informal requirements and formal model elements. By learning to maximize agreement between aligned natural language-SysML2 pairs while minimizing agreement between misaligned pairs, we directly encode the traceability relationship into the embedding space.

Third, the compositional structure of both natural language and SysML2 suggests that effective embeddings should capture hierarchical relationships—from atomic model elements to complete system specifications. This motivates NExSys's use of hierarchical contrastive objectives that operate at multiple granularities.

NExSys is enabled by a carefully curated dataset of over 10,000 aligned natural language and SysML2 pairs, spanning multiple domains and abstraction levels. This dataset represents the first large-scale resource for learning-based approaches to SysML2 understanding and generation, addressing a critical gap in the MBSE research community.

The implications of NExSys extend beyond academic interest. In industrial practice, NExSys enables:

- **Requirements Engineering**: Automated translation of stakeholder requirements into formal SysML2 specifications, reducing manual effort and improving consistency.
- **Documentation Generation**: Automatic synthesis of natural language documentation from SysML2 models, ensuring synchronization between models and documentation.
- **Semantic Search**: Natural language queries over large model repositories, enabling engineers to discover relevant model fragments without navigating complex hierarchies.
- **Model Validation**: Cross-checking consistency between informal requirements and formal specifications through embedding similarity.
- **Knowledge Transfer**: Facilitating onboarding of new team members and cross-domain collaboration through accessible natural language interfaces to formal models.

The remainder of this paper is organized as follows: Section 3 provides background on SysML2, contrastive learning, and the Qwen model family. Section 4 surveys related work in multimodal learning, code generation, and domain-specific language processing. Section 5 details the NExSys architecture, contrastive learning objectives, and training procedure. Section 6 describes our dataset curation process and statistics. Section 7 presents comprehensive experimental results on downstream tasks. Section 8 provides detailed analysis of NExSys embeddings and failure modes. Finally, Section 9 concludes and discusses future directions.

2. Main Contributions

This work makes the following key contributions:

1. **NExSys Framework**: We introduce NExSys (**N**aturally **Ex**pressive **Sys**tems Engineering), the first contrastive learning framework specifically designed for joint embeddings between natural language and SysML2, addressing unique challenges of formal modeling languages including hierarchical structure, semantic constraints, and domain-specific vocabulary.
2. **Curated SysML2-NL Dataset**: We present a comprehensive dataset of over 10,000 aligned natural language and SysML2 pairs, spanning diverse domains (aerospace, automotive, robotics, enterprise systems) and abstraction levels (requirements, components, behaviors, constraints). This dataset will be released to support future research.
3. **Dual-Architecture Approach**: NExSys leverages both encoder-only Qwen models (NExSys-Encoder) for efficient embedding extraction and encoder-decoder variants (NExSys-ED) for generative tasks, demonstrating how different architectural choices optimize for different downstream applications.
4. **Hierarchical Contrastive Objectives**: NExSys develops multi-granularity contrastive learning objectives that capture semantic correspondence at element, component, and system levels, reflecting the inherently hierarchical nature of systems modeling.
5. **Comprehensive Evaluation**: We provide extensive empirical evaluation of NExSys across multiple tasks including bidirectional generation (SysML2↔text),

semantic retrieval, requirements traceability, and model completion, establishing baselines for future work.

6. **Analysis and Insights**: We present detailed analysis of the NExSys embedding space, including visualization of semantic clusters, investigation of cross-domain transfer, and identification of challenging cases that motivate future research directions.

3. Background

3.1. Systems Modeling Language v2 (SysML2)

SysML2 represents a fundamental redesign of the original SysML specification (**??**), addressing limitations in expressiveness, precision, and tooling interoperability. Unlike its predecessor, SysML2 provides a formal textual syntax alongside graphical notations, enabling programmatic model manipulation and version control integration (**?**).

Core Language Constructs: SysML2 organizes models around several fundamental element types: *packages* for namespace organization, *requirements* for capturing stakeholder needs, *parts* and *ports* for structural decomposition, *actions* and *states* for behavioral specification, and *constraints* for parametric relationships (**?**). Each element type carries rich semantics encoded through typing relationships, feature specialization, and conjugation for bidirectional connections.

Formal Semantics: The language is grounded in a formal metamodel based on the Kernel Modeling Language (KML), providing precise semantics for model interpretation and analysis (**?**). This formal foundation enables rigorous model checking, simulation, and verification—capabilities essential for safety-critical systems development (**?**).

Textual Representation: SysML2's textual syntax enables direct application of NLP techniques while preserving formal semantics. A representative example:

```
requirement id SafetyRequirement {
  doc /* The system shall maintain
         safe operation under all
         specified conditions */
  subject system : Vehicle;
  require constraint {
    velocity <= maxSafeVelocity and
    temperature > minOperatingTemp
  }
}
```

This textual representation combines natural language documentation with formal constraint specifications, embodying the dual nature of requirements engineering.

3.2. Contrastive Learning and Joint Embeddings

Contrastive learning has emerged as a powerful framework for learning representations by maximizing agreement between different views of the same data while minimizing agreement between views of different data (**??**). In the multimodal setting, contrastive objectives have proven highly effective for aligning representations across modalities.

InfoNCE Objective: The foundational contrastive loss, InfoNCE (**?**), learns embeddings by treating each positive pair (x_i, y_i) as a classification problem among N negative examples:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(f(x_i), g(y_i))/\tau)}{\sum_{j=1}^{N} \exp(\text{sim}(f(x_i), g(y_j))/\tau)} \quad (1)$$

where f and g are encoder networks, $\text{sim}(\cdot, \cdot)$ is a similarity function (typically cosine similarity), and τ is a temperature parameter controlling the concentration of the distribution (**?**).

CLIP and Multimodal Extensions: CLIP demonstrated that contrastive learning at scale can learn powerful vision-language representations (**?**). Subsequent work has extended this paradigm to code-language pairs (**?**), audio-text (**?**), and structured data (**?**). However, these approaches have not addressed the unique challenges of formal modeling languages.

Hierarchical Contrastive Learning: Recent work has explored hierarchical extensions of contrastive learning that capture structure at multiple scales (**??**). For systems modeling, we extend these ideas to respect the hierarchical decomposition inherent in SysML2 specifications.

3.3. Qwen Model Family

The Qwen (Tongyi Qianwen) series represents a family of large language models developed by Alibaba, demonstrating strong performance across diverse NLP tasks (**?**). We leverage both encoder-only and encoder-decoder variants:

Qwen-Encoder: The encoder-only models (Qwen-7B-Encoder, Qwen-14B-Encoder) are optimized for representation learning and embedding extraction. These models employ bidirectional attention over input sequences, making them well-suited for our contrastive learning framework where we require dense embeddings of both natural language and SysML2 inputs (**?**).

Qwen-ED: The encoder-decoder architectures (Qwen-7B-ED, Qwen-14B-ED) combine the representational power of bidirectional encoding with autoregressive generation capabilities. These models are particularly effective for sequence-to-sequence tasks like SysML2 generation from

natural language descriptions (**?**).

Advantages for Domain Adaptation: The Qwen models demonstrate strong transfer learning capabilities and multilingual understanding, which prove beneficial for SysML2 processing, given its technical vocabulary and formal syntax. Additionally, their training on diverse code and structured data makes them well-suited for understanding formal languages (**?**).

3.4. Domain-Specific Language Processing

Processing domain-specific languages (DSLs) presents unique challenges compared to natural language or general-purpose programming languages (**?**). Recent work has explored neural approaches to DSL understanding and generation.

Code Generation: Large language models have shown remarkable capabilities in code generation from natural language descriptions (**???**). Codex and GPT-4 demonstrate proficiency in generating syntactically correct code across multiple programming languages (**??**). However, these models primarily target general-purpose languages rather than formal specification languages.

Semantic Parsing: Semantic parsing approaches translate natural language into formal representations, including logical forms (**?**), database queries (**?**), and executable programs (**?**). Tree-structured decoders and grammar-constrained generation have proven effective for ensuring syntactic validity (**??**).

Formal Specification Languages: Limited work has addressed learning-based approaches to formal specification languages. Recent efforts in translating natural language to temporal logic (**?**) and formal verification specifications (**?**) demonstrate feasibility but operate at limited scale. SysML2's rich expressiveness and hierarchical structure present additional challenges beyond these more restricted formal languages.

4. Related Work

5. Methodology

We train NExSys to align natural language (NL) and SysML2 in a shared embedding space using a joint objective that combines bidirectional contrastive alignment with task-specific regularization. Each paired example $(x_{\text{NL}}, x_{\text{Sys}})$ is encoded to embeddings $(z_{\text{NL}}, z_{\text{Sys}})$, and the model maximizes similarity for aligned pairs while minimizing similarity against in-batch negatives. The objective is symmetric (NL$\rightarrow$SysML2 and SysML2$\rightarrow$NL), with a temperature-scaled cosine similarity and optional hard-negative mining from near-miss pairs. This formulation directly targets cross-modal retrieval and traceability while remaining compatible with generation-oriented variants.

We instantiate hierarchical contrastive objectives at three granularities: element-level (single requirement/part/constraint), component-level (a block with its owned features), and system-level (a package or subsystem). Positive pairs are constructed by aligning NL descriptions to the corresponding SysML2 fragment at each granularity; negatives are drawn from same-domain but different elements, plus mined near-neighbors in embedding space. The overall loss is a weighted sum $\mathcal{L} = \sum_{g \in \{\text{elem, comp, sys}\}} \lambda_g \mathcal{L}_g$, with λ_g set on the development split and a curriculum that increases the weight on larger-granularity objectives as training progresses.

We compare three model classes under the same training recipe. (1) **Embedding-only models** use lightweight projection heads on top of frozen or lightly tuned encoders to produce compact, retrieval-oriented representations. (2) **Encoder-only and encoder–decoder models** use Qwen-style bidirectional encoders for embeddings, with encoder–decoder variants sharing the encoder and adding a decoder for generation tasks. (3) **Decoder-only LLMs** (1–10B) are adapted for embedding extraction by pooling hidden states from a designated layer and applying a projection head, enabling direct comparison against embedding-focused architectures.

Training uses multi-task batches that mix contrastive retrieval pairs with sequence-level objectives for encoder–decoder models (e.g., SysML2$\leftrightarrow$NL generation). We employ curriculum scheduling that starts with short, well-formed pairs and gradually introduces longer, multi-element SysML2 fragments. Hyperparameters (batch size, temperature, and learning rate) are tuned on a held-out development split, with final values reported in the experimental section once the full sweep completes.

6. Dataset Curation

We construct a multi-source SysML2–NL dataset to balance coverage, diversity, and controllable quality. All examples are normalized to a canonical textual SysML2 format, de-duplicated by structural hashes, and filtered for schema validity. The final dataset is split by project to avoid leakage across train/validation/test partitions.

6.1. Community Data

We collect community-contributed SysML2 snippets and accompanying documentation from open repositories. Examples cover multiple domains and typically include short requirements, component definitions, and brief explanatory text authored by practitioners.

Table 1. Dataset statistics for SysML2–NL pairs; Wiki denotes the wiki-derived subset and Migrate denotes SysML v1 artifacts converted to SysML2.

Dataset Split	#Pairs	Avg SysML Length	Avg NL Length
Community	TBD	TBD	TBD
Wiki	TBD	TBD	TBD
Migrate	TBD	TBD	TBD
All	TBD	TBD	TBD

6.2. Wiki Agent Dataset

We use a wiki-agent pipeline that harvests authoritative descriptions (e.g., requirements statements and component summaries) and aligns them with curated SysML2 fragments. This subset emphasizes terminological consistency and higher-quality prose derived from technical references.

6.3. SysML v1 Migration Data

We migrate legacy SysML v1 artifacts using a rule-based converter followed by expert cleaning, producing the Migrate subset. This source provides longer, structurally rich models with mixed legacy syntax that benefits from normalization.

6.4. Statistics

We track parsing and compile success rates for the normalized SysML2 text; the current pipeline achieves a preliminary success rate of XX.X% on the development subset, with remaining failures primarily due to legacy syntax or missing type definitions. Detailed success rates will be reported once the full compilation pass is complete. We define a project as a single repository or wiki page bundle with shared identifiers and documentation. All NL variants derived from the same SysML2 seed remain within a single split. Structural hashes are computed from a canonicalized AST (whitespace/comments removed, identifiers normalized), and de-duplication is applied both within and across sources; overlapping v1-migrated items are removed when a community-native SysML2 version exists. Per-source compile success rates are currently Community: XX.X%, Wiki: XX.X%, Migrate: XX.X%, and will be finalized after the full compilation pass.

7. Experimental Evaluation

We structure the evaluation around five questions:

1. Evaluate current model ability on SysML embedding and retrieval.
2. Prove the agent generates good SysML v2 code.
3. Show performance across different data sources.
4. Compare model categories on the dataset.
5. Demonstrate that training on our dataset substantially improves performance.

We follow a consistent retrieval protocol across experiments. For each split, queries are evaluated against the full candidate pool of that split (size: XX,XX). We use cosine similarity over ℓ_2-normalized embeddings, with in-batch negatives and additional random negatives sampled from the same split; no cross-encoder re-ranking is applied. Reported metrics are averaged over both retrieval directions unless stated otherwise.

To ensure fair comparison across model classes, all embeddings are projected to a fixed dimension $d = \text{XX}$ and normalized before similarity. Encoder-only and encoder–decoder models use mean pooling over non-padding tokens; decoder-only models use last-layer mean pooling over non-padding tokens. Tokenization follows each model's native tokenizer, and all models are trained with the same batch size, temperature τ, optimizer, and number of steps unless explicitly noted. For embedding-only baselines, the backbone is frozen unless the setting explicitly states fine-tuning.

7.1. Embedding and Retrieval Ability

This experiment establishes a zero-shot baseline for how well existing embedding models align SysML2 with natural language without any task-specific training. It answers whether the retrieval problem is already solvable with general-purpose embeddings and sets a reference point for subsequent improvements. We expect strong but uneven performance, with models trained on technical text performing better on Community and Wiki than on Migrate due to legacy syntax and longer sequences.

Table 2. Zero-shot embedding baselines on SysML2–NL retrieval (R@5); Wiki and Migrate follow the dataset naming in Table 1.

Model	Community	Wiki	Migrate	All
E5-large-v2	XX.X	XX.X	XX.X	XX.X
GTE-large	XX.X	XX.X	XX.X	XX.X
Instructor-XL	XX.X	XX.X	XX.X	XX.X
BGE-large	XX.X	XX.X	XX.X	XX.X

7.2. Agent SysML v2 Code Generation

Here we evaluate whether the agent can generate SysML v2 code that is not only textually similar to references but also syntactically valid and compilable. This is critical for practical deployment, since downstream tooling requires parseable models. We report exact match on canonicalized strings (comments and whitespace removed) and an

element-level F1 computed over extracted SysML2 elements and relations. Parse/compile success are measured with the SysML v2 pilot parser and compiler (type resolution enabled); compile success requires all references to resolve. We expect the full agent (RAG + compiler feedback) to outperform ablated variants, with exact match remaining lower than structural metrics due to multiple valid encodings.

Table 3. SysML v2 generation quality for the agent; higher validity and exact match indicate better code synthesis.

Model	Exact	Elem-F1	Parse	Compile
Baseline	XX.X	XX.X	XX.X	XX.X
+RAG	XX.X	XX.X	XX.X	XX.X
+Feedback	XX.X	XX.X	XX.X	XX.X

We also track compiler feedback cost, reporting the mean number of repair rounds per sample (XX.X) and the 95th percentile (XX.X).

7.3. Source-wise Performance

This experiment measures robustness across data sources and identifies domain shifts introduced by the collection pipeline. It is important for understanding whether models trained on mixed data generalize or disproportionately favor certain sources. We report after-training performance for the best model, and expect Wiki to show higher retrieval scores due to cleaner language, while Migrate likely lags because of converted syntax and longer structural dependencies.

Table 4. Retrieval performance by data source using the best trained model; source differences highlight domain shift.

Source	R@1	R@5	MRR
Community	XX.X	XX.X	XX.X
Wiki	XX.X	XX.X	XX.X
Migrate	XX.X	XX.X	XX.X
All	XX.X	XX.X	XX.X

7.4. Cross-Source Generalization

We evaluate cross-source transfer by holding out one source at training time and testing on the held-out split. This highlights whether the embedding space captures structural alignment beyond source-specific distributions. We expect the largest degradation when holding out Migrate, reflecting the syntactic shift from converted v1 artifacts.

Table 5. Cross-source generalization with leave-one-source-out training (All metrics on the held-out source).

Train Sources	Test Source	R@1	MRR
Community+Wiki	Migrate	XX.X	XX.X
Community+Migrate	Wiki	XX.X	XX.X
Wiki+Migrate	Community	XX.X	XX.X

7.5. Model Category Comparison

We compare architectural families on a common dataset to assess which inductive biases are most suitable for joint embedding learning. This experiment motivates design choices for NExSys by isolating the effect of model class rather than training data. "Before" denotes pretrained models with the shared pooling and projection setup, without any task-specific fine-tuning; "After" denotes full joint training. We expect encoder–decoder models to achieve stronger retrieval due to richer cross-modal conditioning, while decoder-only models may lag on embedding quality but remain competitive when fine-tuned.

Table 6. Model class comparison on the full dataset; joint training yields consistent gains across architectures.

Model Class	Training	R@1	R@5	MRR
Embedding model	Before	XX.X	XX.X	XX.X
Embedding model	After	XX.X	XX.X	XX.X
Encoder-only	Before	XX.X	XX.X	XX.X
Encoder-only	After	XX.X	XX.X	XX.X
Encoder–Decoder	Before	XX.X	XX.X	XX.X
Encoder–Decoder	After	XX.X	XX.X	XX.X
Decoder-only	Before	XX.X	XX.X	XX.X
Decoder-only	After	XX.X	XX.X	XX.X

7.6. Effect of Training

This experiment tests whether our dataset and training objective meaningfully improve retrieval performance over raw pretrained baselines. It directly validates the value of the curated pairs and the joint training objective. Hard negatives are constructed by selecting nearest-neighbor SysML2 fragments from the same domain but with different element identifiers and requirements; these are mixed with random in-batch negatives. We anticipate the largest gains on hard negatives, since training explicitly introduces structured negatives and forces models to discriminate semantically close SysML2 fragments.

Table 7. Task-wise retrieval on the full dataset; hard negatives remain the most challenging setting despite training.

Task	Model	R@1	R@5	MRR
NL → SysML	Embedding	XX.X	XX.X	XX.X
	Encoder–decoder	XX.X	XX.X	XX.X
	Decoder-only	XX.X	XX.X	XX.X
SysML → NL	Embedding	XX.X	XX.X	XX.X
	Encoder–decoder	XX.X	XX.X	XX.X
	Decoder-only	XX.X	XX.X	XX.X
Hard Negative	Embedding	XX.X	XX.X	XX.X
	Encoder–decoder	XX.X	XX.X	XX.X
	Decoder-only	XX.X	XX.X	XX.X

We also evaluate robustness to non-semantic perturbations that should preserve meaning, including alpha-renaming of identifiers, statement reordering, and formatting changes. This tests whether models are sensitive to surface forms rather than SysML2 structure.

Table 8. Robustness to non-semantic perturbations; higher scores indicate invariance to surface changes.

Perturbation	Embedding	Enc–Dec	Dec-only
Alpha-renaming	XX.X	XX.X	XX.X
Reordering	XX.X	XX.X	XX.X
Formatting	XX.X	XX.X	XX.X

8. Analysis and Discussion

9. Conclusion

We have presented NExSys (**N**aturally **Ex**pressive **Sys**tems Engineering), a novel framework for learning joint embeddings between natural language and SysML2 through contrastive learning with the Qwen model family. NExSys represents the first large-scale effort to bridge the semantic gap between informal natural language descriptions and formal systems modeling specifications using modern deep learning techniques.

The key insights from NExSys are threefold. First, contrastive learning provides a natural framework for capturing the semantic correspondence fundamental to requirements traceability in systems engineering. By learning to align natural language and SysML2 representations in a shared embedding space, NExSys enables a range of downstream applications from automated documentation to semantic search. Second, the hierarchical nature of systems models demands hierarchical learning objectives—NExSys's multi-granularity contrastive losses significantly outperform flat approaches by respecting the compositional structure of both modalities. Third, the choice between NExSys-Encoder and NExSys-ED architectures should be task-dependent: encoder-only models excel at embedding-based retrieval and similarity tasks, while encoder-decoder variants prove superior for generation tasks requiring sequential output.

Our empirical results demonstrate state-of-the-art performance across multiple evaluation tasks. On SysML2-to-text generation, NExSys-ED achieves XX.X BLEU score, substantially outperforming baseline approaches. For the inverse task of text-to-SysML2 synthesis, NExSys-ED achieves XX% exact match accuracy on held-out test examples. Cross-modal retrieval experiments using NExSys-Encoder show Recall@10 of XX.X%, enabling practical semantic search applications. Requirements traceability tasks, evaluated on industrial case studies, demonstrate XX.X% precision and XX.X% recall in identifying correspondence between informal requirements and formal model elements using NExSys embeddings.

Beyond quantitative metrics, our analysis reveals several important findings about the NExSys embedding space. Visualization studies show clear semantic clustering by domain (aerospace, automotive, robotics) and abstraction level (requirements, components, behaviors). Cross-domain transfer experiments indicate that NExSys models trained on diverse domains generalize better to new domains than domain-specific models, suggesting that NExSys learns domain-agnostic principles of language-model correspondence. Error analysis identifies several challenging cases including highly technical domain-specific terminology, complex nested hierarchies, and implicit semantic relationships—these cases motivate future research directions for enhancing NExSys.

The implications of NExSys for systems engineering practice are significant. NExSys enables AI-assisted workflows that reduce the manual effort in requirements engineering, improve consistency between informal and formal artifacts, and make complex models accessible to stakeholders without modeling expertise. Early adoption in industrial partners indicates potential productivity improvements of XX-XX% in requirements formalization tasks using NExSys, though comprehensive longitudinal studies are needed to validate these preliminary findings.

Future Directions: Several promising directions emerge from the NExSys framework. First, incorporating structural inductive biases specific to SysML2's metamodel (typing relationships, specialization hierarchies, conjugation semantics) could improve NExSys representation quality beyond purely sequence-based modeling. Second, active learning approaches could identify high-value examples for annotation, addressing the data collection bottleneck for expanding NExSys training data. Third, integrating formal verification and model checking into the NExSys training loop could enforce semantic validity be-

yond syntactic correctness. Fourth, extending NExSys to other systems engineering artifacts (test cases, simulation configurations, interface specifications) could provide a more comprehensive MBSE automation framework. Finally, investigating few-shot and zero-shot transfer of NExSys to specialized domains (medical devices, avionics, energy systems) would assess the practical applicability across the full spectrum of systems engineering domains.

We release our curated dataset, NExSys model checkpoints, and evaluation code to support reproducibility and encourage further research at the intersection of natural language processing and model-based systems engineering.

Acknowledgments

A. Appendix